Análisis de maniobra de asistencia gravitacional

Mecánica Clásica I

Eneko Acevedo Pérez

Índice

1. Introducción **4**

1.1. Plan de vuelo 4

I Cálculo Analítico 5

1. La transferencia heliocéntrica **5**

2. Dispersión en la Esfera de Influencia (SOI) **6**

3. Código **7**

3.1. Cálculo del parámetro de impacto a partir de la fase 7

3.2. Simulación de la trayectoria de asistencia 8

3.3. Optimización iterativa 10

II Simulación Numérica de N Cuerpos 12

1. Lagrangiano del sistema **12**

2. Integración temporal y paso adaptativo **12**

3. Validación mediante leyes de conservación **13**

4. Código **13**

4.1. Cálculo de aceleraciones gravitatorias 14

4.2. Paso de integración: Runge–Kutta 4 14

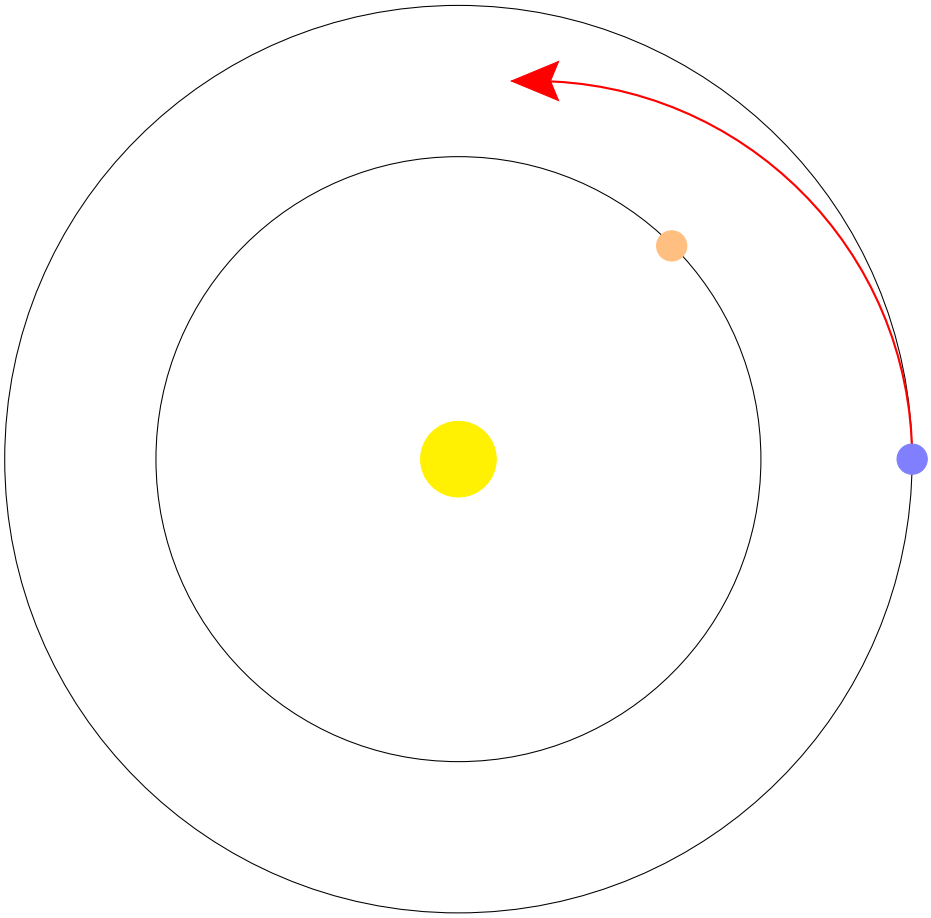
4.3. Cálculo del Hamiltoniano (energía total) y energía de la nave 15

4.4. Bucle principal y ajuste de paso temporal 16

III Análisis de resultados 17

1. Aproximación por el método analítico **17**

Análisis de maniobra de asistencia gravitacional



1. Introducción

El presente trabajo aborda la simulación de un sistema solar reducido compuesto por el Sol, la Tierra, Venus y una nave espacial.

El objetivo central del estudio es el análisis y optimización de la maniobra de asistencia gravitacional. Se examina el intercambio de momento angular entre el planeta y la nave para determinar las condiciones iniciales de lanzamiento y el parámetro de impacto que maximizan la velocidad heliocéntrica final de la sonda. Se construye una superficie de optimización, mutando las variables que afectan al intercambio de energía, que permite identificar la trayectoria de máxima ganancia.

1.1. Plan de vuelo

Se va a proceder con dos fases para la rutina de cálculo.

Como primera tenemos el método analítico. El Sol se considerará fijo en toda la trayectoria de estudio. Al principio de la órbita de transferencia, solo se considerará la acción del Sol sobre la nave para obtener una elíptica dependiente de la velocidad inicial. Con esto se tendrán dos puntos de corte con la órbita de Venus para cada trayectoria. Para cada punto de corte, se modelará el sistema Venus nave variando la posición de Venus en un entorno centrado en el corte.

El punto exacto en que se cambia de sistema de referencia queda determinado por el radio de la esfera de influencia gravitatoria de Venus (Weber, 2025).

Después de la trayectoria hiperbólica y al salir de radio de influencia de Venus, se volverá al sistema de referencia heliocéntrico para medir su hamiltoniano. Con esto se tiene una forma de evaluar la energía transferida a la nave, que es la cantidad a maximizar.

Finalmente, se hará un barrido que analice todo el espacio de condiciones iniciales para determinar las ventanas de condiciones iniciales que resulten en máximos locales.

La segunda parte es numérica. Se simulará computacionalmente el sistema nave, Venus y Sol, utilizando un integrador RK4 con paso temporal adaptativo (dependiente de la distancia de la nave al cuerpo más cercano) para salvar singularidades y mejorar la precisión. Con este modelo de asumida suficiente fidelidad, se analizarán las ventanas de condiciones iniciales que se extrayeron en la primera fase.

Parte I

Cálculo Analítico

En esta parte del trabajo abordamos el problema dividiendo la trayectoria en dos regímenes dinámicos diferenciados, una técnica llamada aproximación por secciones cónicas. Para cada régimen, simplificamos el sistema general de N cuerpos reduciéndolo a problemas locales de dos cuerpos, lo cual nos permite aplicar soluciones analíticas exactas.

1. La transferencia heliocéntrica

Para la fase de crucero entre la Tierra y Venus, consideramos el sistema Sol-nave aislado. Según la teoría de fuerzas centrales, este sistema de dos cuerpos puede reducirse a un problema efectivo de un solo cuerpo de masa reducida μ moviéndose en un potencial central $V(r)$ (Subías, 2025).

Dado que la masa de la nave, m, es despreciable frente a la del Sol ($m \ll M_\odot$), la masa reducida $\mu \approx m$ y el centro de masas coincide prácticamente con el Sol. La dinámica queda gobernada por la conservación de la energía mecánica E y del momento angular $\vec{L}$, lo que confina la trayectoria a un plano invariante. Por tanto, podemos describir su órbita con:

$$r = \frac{p}{1 + e\cos\theta} \tag{1}$$

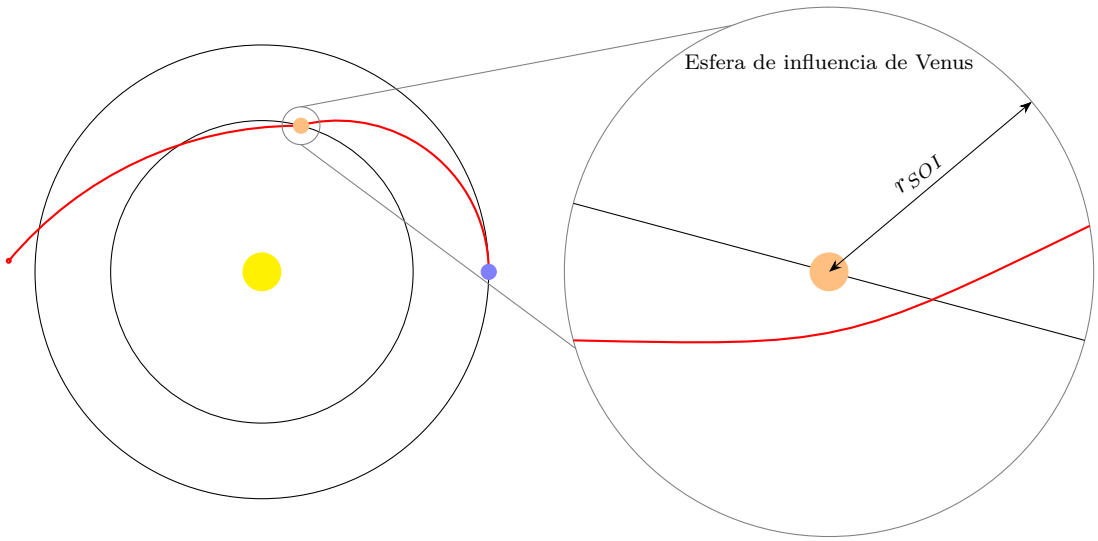
siendo r la distancia relativa en cierto instante, p el parámetro de la órbita, e la excentricidad y θ el ángulo recorrido hasta el mismo instante.

La ecuación de la vis-viva, derivada directamente de la integral de la energía para el potencial gravitatorio newtoniano, determina la velocidad en cualquier punto de la elipse de transferencia:

$$v^2 = \mu_\circ \left(\frac{2}{r} - \frac{1}{a}\right) \tag{2}$$

donde v es la velocidad orbital del cuerpo que orbita, a el semieje mayor de la elipse que describe (definido por la energía total negativa del sistema, $E < 0$), r la distancia relativa entre ambos y $\mu_\circ = GM_\circ$ es el parámetro gravitacional estándar del cuerpo al que se orbita.

2. Dispersión en la Esfera de Influencia (SOI)



Cuando la nave ingresa en la Esfera de Influencia de Venus ($r < r_{\text{SOI}}$), realizamos un cambio de sistema de referencia. El problema se modela ahora como la interacción de una partícula libre con un centro dispersor fijo (Venus). Definimos el límite de esta interacción mediante la siguiente aproximación tomada de Weber, 2025:

$$r_{\text{SOI}} \approx r_{\text{v}} \left(\frac{M_{\text{v}}}{M_{\odot}} \right)^{2/5} \tag{3}$$

donde r_{v} es la distancia del Sol a Venus, $M_{\odot}$ es la masa del Sol y M_{v} la masa de Venus.

En este marco local, la nave posee una energía mecánica positiva respecto a Venus ($E > 0$), lo que resulta en una trayectoria abierta hiperbólica. El objetivo de la maniobra se reinterpreta como un problema de dispersión: buscamos maximizar la deflexión del vector velocidad asintótico.

La geometría del encuentro viene caracterizada por el parámetro de impacto s (distancia perpendicular asintótica al centro de fuerzas). Según la teoría de dispersión en campos centrales conservativos, el ángulo de dispersión θ (el ángulo total que gira la velocidad de la nave) está relacionado unívocamente con s mediante la expresión (Subías, 2025):

$$s = \frac{k}{2E} \cot\left(\frac{\theta}{2}\right) \quad \Longrightarrow \quad \theta = 2\tan^{-1}\left(\frac{k}{2Es}\right) \tag{4}$$

donde $k = GM_{\text{v}}m$ es la constante del potencial gravitatorio y $E = \frac{1}{2}mv_{\infty}^2$ es la energía cinética incidente, que será aproximada a la velocidad relativa con respecto a Venus cuando la nave entre en su esfera

de influiencia. Esta relación analítica nos permite apuntar con precisión: variando ligeramente s en la simulación, controlamos directamente el ángulo de salida y, por tanto, la ganancia de momento lineal en el sistema heliocéntrico.

3. Código

3.1. Cálculo del parámetro de impacto a partir de la fase

Esta función estima el parámetro de impacto b en el encuentro con Venus a partir de la velocidad de lanzamiento y la configuración geométrica inicial entre la Tierra y Venus. El objetivo es traducir un desfase angular en una distancia de aproximación.

Listing 1: Cálculo de elementos orbitales y anomalía de llegada.

```
def calcular_b_desde_fase(v_lanzamiento, fase_inicial_deg):
    energia = (v_lanzamiento**2) / 2 - MU_SUN / R_TIERRA if energia >= 0: return None

    a = -MU_SUN / (2 * energia)
    h = R_TIERRA * v_lanzamiento
    p = h**2 / MU_SUN

    e_val = np.sqrt(max(0, 1 - p/a))
    cos_nu_llegada = np.clip((p / R_VENUS_ORBIT - 1) / e_val, -1, 1)
    nu_llegada = 2 * np.pi - np.arccos(cos_nu_llegada)
    ...
```

Se determinan las constantes de la órbita de transferencia (energía, semieje mayor y excentricidad) asumiendo un lanzamiento desde la posición de la Tierra. Se calcula la anomalía verdadera en el punto de intersección con la órbita de Venus, seleccionando el nodo correspondiente a la trayectoria de llegada.

Listing 2: Determinación del tiempo de vuelo mediante la ecuación de Kepler.

```
def calcular_b_desde_fase(v_lanzamiento, fase_inicial_deg):
    ...
    angulo_recorrido_sc = nu_llegada - np.pi

    E_inicio = np.pi
    E_llegada = 2 * np.pi - 2 * np.arctan(np.sqrt((1-e_val)/(1+e_val)) * np.tan((2*np.pi -
    nu_llegada)/2))

    M_inicio = E_inicio - e_val * np.sin(E_inicio)
    M_llegada = E_llegada - e_val * np.sin(E_llegada)

    n = np.sqrt(MU_SUN / a**3)
    t_vuelo = (M_llegada - M_inicio) / n
    ...
```

Se calcula el tiempo de vuelo necesario para que la sonda recorra la trayectoria elíptica. Para ello, se transforman las anomalías verdaderas en anomalías excéntricas y, posteriormente, en anomalías medias

a través de la ecuación de Kepler (tomado de García, 2016), permitiendo obtener el tiempo exacto de tránsito.

Listing 3: Cálculo del error angular y parámetro b.

```
def calcular_b_desde_fase(v_lanzamiento, fase_inicial_deg):
    ...
    omega_v = np.sqrt(MU_SUN / R_VENUS_ORBIT**3) fase_inicial_rad = np.radians(
    fase_inicial_deg) angulo_venus_final = fase_inicial_rad + (omega_v * t_vuelo)

    error_angular = angulo_recorrido_sc - angulo_venus_final
    error_angular = (error_angular + np.pi) % (2 * np.pi) - np.pi

    b = R_VENUS_ORBIT * np.sin(error_angular)
    return b
```

Finalmente, se calcula la posición de Venus tras el tiempo de vuelo y se compara con la posición de llegada de la sonda. El desfase angular resultante (normalizado entre $-\pi$ y π) se proyecta sobre el radio orbital de Venus para obtener el parámetro de impacto b, que define la geometría del encuentro.

3.2. Simulación de la trayectoria de asistencia

Esta función constituye el núcleo físico del simulador. Calcula la interacción gravitatoria entre la sonda y Venus, determinando cómo varía la energía heliocéntrica tras el encuentro mediante el modelo de parches de cónicas.

Listing 4: Cálculo del encuentro y velocidad de exceso.

```
def simulate_analytic_flyby(v_launch_mag, phase, r_launch = 149.6e9, debug=False):
    impact_parameter_b = calcular_b_desde_fase(v_launch_mag, phase)
    if np.isnan(impact_parameter_b):
        return np.nan

    h = r_launch * v_launch_mag
    energy_initial = (v_launch_mag**2)/2 - MU_SUN/r_launch
    a = -MU_SUN / (2 * energy_initial)
    e = np.sqrt(1 + (2 * energy_initial * h**2) / (MU_SUN**2))
    p = a * (1 - e**2)

    cos_nu = (p / R_VENUS_ORBIT - 1) / e
    nu_intersect = -np.arccos(np.clip(cos_nu, -1, 1))

    v_perp = MU_SUN / h * (1 + e * np.cos(nu_intersect))
    v_radial = MU_SUN / h * e * np.sin(nu_intersect)
    phi_sc = np.arctan2(v_radial, v_perp)
    v_sc_mag = np.sqrt(v_perp**2 + v_radial**2)

    v_venus_mag = np.sqrt(MU_SUN / R_VENUS_ORBIT)
    v_inf_sq = v_sc_mag**2 + v_venus_mag**2 - 2 * v_sc_mag * v_venus_mag * np.cos(phi_sc)
    v_inf_mag = np.sqrt(max(0, v_inf_sq))
    ...
```

Se determinan las condiciones cinemáticas justo antes del encuentro. Se calcula el ángulo de la trayectoria

(φ_{sc}) y la velocidad relativa o de exceso hiperbólico v_∞ mediante el teorema del coseno, combinando los vectores de velocidad de la sonda y de Venus en el punto de intersección de sus órbitas.

Listing 5: Análisis de la hipérbola y control de colisión.

```
def simulate_analytic_flyby(v_launch_mag, phase, r_launch = 149.6e9, debug=False):
    ...
    if abs(impact_parameter_b) < 1.0:
        impact_parameter_b = 1.0 if impact_parameter_b >= 0 else -1.0

    term_hyp = (impact_parameter_b * v_inf_sq) / MU_VENUS
    e_hyp = np.sqrt(1 + term_hyp**2)
    rp = (MU_VENUS / v_inf_sq) * (e_hyp - 1)

    if rp < SAFE_RADIUS:
        return np.nan

    tan_half_delta = MU_VENUS / (impact_parameter_b * v_inf_sq)
    delta = 2 * np.arctan(tan_half_delta)
    ...
```

Se modela la trayectoria cercana como una hipérbola centrada en Venus. Se calcula el radio de periapsis r_p para asegurar que la sonda no impacte con el planeta o entre en una zona de frenado atmosférico no deseada, comparándolo con el radio seguro (radio de Venus teniendo en cuenta su atmósfera). Si la trayectoria es segura, se obtiene el ángulo de deflexión δ, que representa cuánto se desviará el vector de velocidad de exceso.

Listing 6: Rotación de velocidad y energía final.

```
def simulate_analytic_flyby(v_launch_mag, phase, r_launch = 149.6e9, debug=False):
    ...
    vec_v_venus = np.array([v_venus_mag, 0]) vec_v_sc = np.array([v_sc_mag * np.cos(phi_sc),
    v_sc_mag * np.sin(phi_sc)]) vec_v_inf_in = vec_v_sc - vec_v_venus

    c, s = np.cos(delta), np.sin(delta)
    R = np.array(((c, -s), (s, c)))
    vec_v_inf_out = np.dot(R, vec_v_inf_in)

    vec_v_sc_out = vec_v_venus + vec_v_inf_out
    v_final_mag = np.linalg.norm(vec_v_sc_out)

    e_final = (v_final_mag**2)/2 - MU_SUN/R_VENUS_ORBIT
    return e_final
```

En la fase final, se aplica la matriz de rotación al vector de velocidad de entrada para obtener la dirección de salida. Al sumar vectorialmente la velocidad orbital de Venus, se recupera la velocidad heliocéntrica resultante. El valor de retorno es la energía específica final (Hamiltoniano), cuya optimización permitirá maximizar el cambio de órbita deseado.

3.3. Optimización iterativa

Esta es la función que hará el barrido del espacio de condiciones iniciales. Toma rangos de ambas condiciones, hace una malla de valores del hamiltoniano final y repite en un rango más pequeño y centrado en el máximo tantas veces como pasos se hayan elegido.

Listing 7: Inicialización y comienzo del bucle.

```
def iterative_optimization(v_center, v_span, b_center, b_span, steps, grid_res):
    best_v = 0
    best_b = 0
    max_e = -np.inf

    print(f"--- INICIANDO OPTIMIZACIÓN ITERATIVA ({steps} PASOS) ---")

    for step in range(steps):
        v_min, v_max = v_center - v_span/2, v_center + v_span/2
        b_min, b_max = b_center - b_span/2, b_center + b_span/2

        v_vals = np.linspace(v_min, v_max, grid_res)
        b_vals = np.linspace(b_min, b_max, grid_res)

        energy_grid = np.zeros((grid_res, grid_res))
        ...
```

Se inicializan las variables cuyo máximo se quiere encontrar y sus rangos y además de la malla solución del paso actual dentro del bucle principal. El bucle principal hace una iteración por cada reducción de los rangos.

Listing 8: Iteración y cálculo.

```
def iterative_optimization(v_center, v_span, b_center, b_span, steps=5, grid_res=60):
        ...
        for i, b in enumerate(b_vals):
            for j, v in enumerate(v_vals):
                val = simulate_analytic_flyby(v, b)
                energy_grid[i, j] = val if not np.isnan(val) else -np.inf

        if np.all(energy_grid == -np.inf):
            print(f"Paso {step+1}: No se encontraron trayectorias seguras en este rango.")
            break

        idx = np.unravel_index(np.argmax(energy_grid), energy_grid.shape)

        current_max_e = energy_grid[idx]
        current_best_b = b_vals[idx[0]]
        current_best_v = v_vals[idx[1]]

        print(f"Paso {step+1}:")
        print(f"  Rango V: [{v_min/1000:.1f}, {v_max/1000:.1f}] km/s")
        print(f"  Rango b: [{b_min/1e6:.1f}, {b_max/1e6:.1f}] Mm")
        print(f"  --> Óptimo local: V={current_best_v/1000:.4f} km/s, ")
        print(f"        b={current_best_b/1e6:.4f} Mm, E={current_max_e:.2e}")
        ...
```

Se usa la función anterior, que realiza la simulación y devuelve el hamiltoniano final, para cada par de

condiciones iniciales. Ese valor se guarda en la malla y, tras todas las simulaciones, se extrae el valor máximo. Se muestra por pantalla el resumen de los datos de las simulaciones.

Listing 9: Actulización y resultados.

```
def iterative_optimization(v_center, v_span, b_center, b_span, steps=5, grid_res=60):
        ...
        v_center = current_best_v
        b_center = current_best_b

        v_span *= 0.2
        b_span *= 0.2

    print("\n--- OPTIMIZACIÓN COMPLETADA ---")
    print(f"Precisión final V: +/- {v_span/2:.2f} m/s")
    print(f"Precisión final b: +/- {b_span/2000:.2f} km")
```

Se actualizan los valores del centro, que pasa a ser el par de coordenadas con el valor máximo en la malla, y del rango, que pasa a ser un quinto del valor anteior. Finalmente, al acabar el bucle, se muestran los resultados de todo el proceso.

Parte II

Simulación Numérica de N Cuerpos

En esta parte, en lugar de "apagar" y "encender" la gravedad de los distintos cuerpos, simulamos la evolución simultánea de todo el sistema (Sol, Tierra, Venus y nave) integrando directamente las ecuaciones del movimiento. Esto permite capturar las perturbaciones gravitatorias continuas que el método analítico ignora.

1. Lagrangiano del sistema

Para un sistema de N partículas interactuando gravitatoriamente, el estado dinámico queda definido por el Lagrangiano $L = T - V$ (Subías, 2025), donde T es la energía cinética total y V la energía potencial del sistema:

$$L = \sum_{i=1}^{N} \frac{1}{2} m_i |\dot{\vec{r}}_i|^2 + \sum_{i<j} \frac{G m_i m_j}{|\vec{r}_i - \vec{r}_j|} \tag{5}$$

Aplicando las ecuaciones de Lagrange $\frac{d}{dt}\frac{\partial L}{\partial \dot{q}_j} - \frac{\partial L}{\partial q_j} = 0$ a cada coordenada de la nave (también para el resto de cuerpos), obtenemos la ecuación diferencial vectorial acoplada que gobierna su aceleración:

$$\ddot{\vec{r}}_{nave} = - \sum_{j \neq nave} G m_j \frac{\vec{r}_{nave} - \vec{r}_j}{|\vec{r}_{nave} - \vec{r}_j|^3} \tag{6}$$

Esta ecuación no tiene solución analítica cerrada para $N > 2$, por lo que recurrimos a la integración numérica mediante el método de Runge-Kutta de cuarto orden (RK4).

2. Integración temporal y paso adaptativo

Hay que tener en cuenta que el término de energía potencial efectiva diverge cuando la distancia relativa tiende a cero ($V \propto -1/r$).

Durante la maniobra, la nave pasa muy cerca de Venus (en el periapsis), donde la curvatura de la trayectoria y la magnitud de la aceleración cambian drásticamente. Un paso de tiempo fijo (Δt) sería computacionalmente ineficiente en el espacio interplanetario (demasiado pequeño) e impreciso en

el encuentro cercano (demasiado grande).

Para solucionar esto, se implementa un algoritmo de paso de tiempo adaptativo. La simulación ajusta dinámicamente Δt en función de la distancia al cuerpo más cercano ($r_{\min}$):

$$\Delta t = \eta \cdot r_{\min} \tag{7}$$

donde η es un factor de escala empírico. Esto asegura que la resolución temporal aumente automáticamente en las regiones donde el gradiente del potencial es mayor.

3. Validación mediante leyes de conservación

Dado que el Lagrangiano del sistema no depende explícitamente del tiempo ($\partial L/\partial t = 0$), la energía mecánica total (Hamiltoniano) debe conservarse (Subías, 2025):

$$H = \sum_i \frac{|\vec{p}_i|^2}{2m_i} + V(\vec{r}_1, ..., \vec{r}_N) = \text{cte} \tag{8}$$

Utilizamos esta propiedad teórica como herramienta de control de calidad. Monitorizando la variación de la energía total del sistema durante la simulación, verificamos que el error numérico introducido por el integrador RK4 se mantiene acotado, validando así la precisión física de la trayectoria obtenida.

4. Código

En lo que sigue se describen las funciones principales del módulo `integrator.py` usado en la simulación numérica. La explicación se organiza siguiendo el orden del fichero: funciones auxiliares, cálculo de aceleraciones, integrador RK4, manejo del vector-estado, cálculo de energías, bucle de integración con paso temporal adaptativo, salida gráfica y rutinas de optimización.

4.1. Cálculo de aceleraciones gravitatorias

Listing 10: Cálculo de la aceleración neta sobre cada cuerpo por interacción mutua.

```
def calcularAceleraciones(posiciones,masas,G):
    N=len(masas)
    dim = posiciones.shape[1]
    matrizDistancias = np.zeros((N, dim, N))
    for i in range(N):
        for j in range(N):
            if i!=j:
                deltaR = vecRelativo(posiciones[i],posiciones[j])
                matrizDistancias[i,:,j]=unitario(deltaR)/np.linalg.norm(deltaR)**2
    aceleraciones = matrizDistancias @ masas * G
    return aceleraciones
```

La función construye una estructura que para cada par (i, j) contiene el vector unitario desde i hacia j dividido por r^2 (la dependencia típica de la aceleración gravitatoria). Se evita la autointeracción con la condición `if i!=j`. Finalmente, un producto matricial con el vector de masas y el factor gravitatorio `G` produce la aceleración neta sobre cada cuerpo: esto está pensado para explotar la multiplicación lineal y mantener el código compacto.

4.2. Paso de integración: Runge–Kutta 4

Listing 11: Paso RK4 para posiciones y velocidades.

```
def stepRungeKutta4(posiciones,velocidades,masas,G,dt):
    k1_v = calcularAceleraciones(posiciones, masas, G)
    k1_r = velocidades
    k2_v = calcularAceleraciones(posiciones + k1_r * 0.5 * dt, masas, G)
    k2_r = velocidades + k1_v * 0.5 * dt
    k3_v = calcularAceleraciones(posiciones + k2_r * 0.5 * dt, masas, G)
    k3_r = velocidades + k2_v * 0.5 * dt
    k4_v = calcularAceleraciones(posiciones + k3_r * dt, masas, G)
    k4_r = velocidades + k3_v * dt
    nextVels = velocidades + (dt / 6.0) * (k1_v + 2*k2_v + 2*k3_v + k4_v)
    nextPoss = posiciones + (dt / 6.0) * (k1_r + 2*k2_r + 2*k3_r + k4_r)
    return nextPoss,nextVels
```

Se implementa un integrador de cuarto orden que actualiza simultáneamente posiciones y velocidades. La función espera los estados en un determinado instante de timepo y devuelve los nuevos estados tras un paso `dt`. El uso de RK4 proporciona precisión suficiente para las escalas de tiempo y velocidades consideradas, manteniendo estabilidad fuera de las cercanías de las singularidades.

4.3. Cálculo del Hamiltoniano (energía total) y energía de la nave

Listing 12: Cálculo de energía total y energía específica de la nave.

```
def calculoHamiltoniano(posiciones,velocidades,masas,G):
    masas = np.array(masas)
    r_vec = posiciones[:, np.newaxis, :] - posiciones[np.newaxis, :, :]
    r_matrix = np.linalg.norm(r_vec, axis=2)
    with np.errstate(divide='ignore'):
        inv_r = 1.0 / r_matrix
    np.fill_diagonal(inv_r, 0)
    potencialTerm = 0.5 * np.sum(masas[:, None] * inv_r * masas[None, :])
    V = -G * potencialTerm
    v_sq = np.sum(velocidades**2, axis=1)
    T = 0.5 * np.sum(masas * v_sq)
    return T + V

def calculoHamiltonianoNave(posiciones,velocidades,masas,G):
    nave = posiciones[-1]
    otros = posiciones[:-1]
    distancias = np.linalg.norm(otros - nave, axis=1)
    distancias[distancias < 1e-10] = 1e-10
    V = - G * masas[-1] * (1/distancias) @ masas[:-1]
    velNave = velocidades[-1]
    T = masas[-1] * velNave.T @ velNave / 2
    return T + V
```

Ambas rutinas calculan la energía total del sistema y la energía mecánica específica de la nave respecto del sistema heliocéntrico. En `calculoHamiltoniano` se evita la doble cuenta en el potencial con el factor 0.5 y se anula la diagonal de la matriz inversa de distancias para evitar autointeracción. `calculoHamiltonianoNave` evalúa únicamente la contribución potencial debida a los otros cuerpos y la energía cinética de la nave.

4.4. Bucle principal y ajuste de paso temporal

Listing 13: Bucle de integración adaptativa y almacenamiento de historial (resumen).

```
def calculoOrbitas(initPos,initVel,masas,tiempoSim,dtFactor,G,fase,debug=False):
    N=len(masas)
    dim = initPos.shape[1]
    H = calculoHamiltoniano(initPos,initVel,masas,G)
    naveH = calculoHamiltonianoNave(initPos,initVel,masas,G)
    historial = [[0,H,naveH,*initPos.flatten(),*initVel.flatten()]]
    stepNumber = 0
    timePassed = 0
    while timePassed < tiempoSim:
        vectorEstado = historial[-1]
        velActual = extraerVelocidadesVectorEstado(vectorEstado,dim,N)
        posActual = extraerPosicionesVectorEstado(vectorEstado,dim,N)
        velNaveNorm = (velActual[-1] @ velActual[-1])**.5
        rmin = distanciaMinimaUltimoCuerpo(posActual)
        if rmin*dtFactor > 1e2:
            dt = dtFactor*rmin/velNaveNorm
        else:
            dt = 100.0
        if rmin < 6151800:
            print("\n====================\nHa habido una colisión.\n====================")
            return np.array([0])
        nextPos, nextVel = stepRungeKutta4(posActual,velActual,masas,G,dt)
        stepNumber += 1
        timePassed += dt
        H = calculoHamiltoniano(nextPos,nextVel,masas,G)
        naveH = calculoHamiltonianoNave(nextPos,nextVel,masas,G)
        historial.append([timePassed,H,naveH, *nextPos.flatten(), *nextVel.flatten()])
    return np.array(historial)
```

El bucle central itera hasta cubrir el tiempo total `tiempoSim`. El paso de tiempo se adapta en función de la distancia mínima de la nave al resto de cuerpos: se escala proporcionalmente a `rmin` y al factor de escalado `dtFactor`, normalizado por la velocidad de la nave para mantener resolución espacial aproximada. Si la nave entra en una zona crítica (radio inferior a 6.151.800 m) se considera colisión y la rutina termina. En cada paso se actualizan las energías y se almacena el estado completo en `historial`, cuya estructura permite un análisis posterior y la generación de gráficas.

Parte III

Análisis de resultados

1. Aproximación por el método analítico

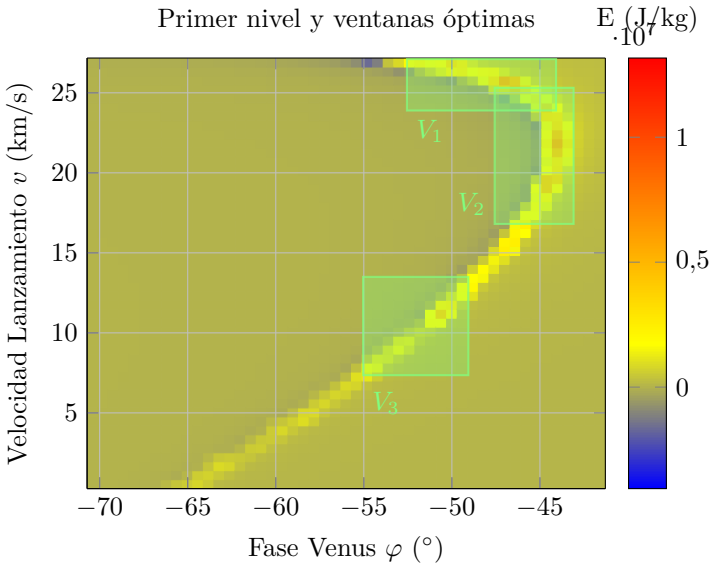


Figura 1: Mapa de energía obtenido mediante optimización iterativa.

En el gráfico[1] están marcadas en verde las ventanas (V_1, V_2 y V_3) consideradas relevantes. Para las siguiente iteración tomamos los intervalos mostrados en la siguiente tabla.

	v (km/s)	φ (°)
V_1	$[24, 28]$	$[-53, -44]$
V_2	$[12, 26]$	$[-47, -43]$
V_3	$[8, 12]$	$[-55, -48]$

[1] Para disminuir el coste del renderizado, se ha tomado la media de la potencia cuarta de grupos de la malla real para reducir la resolución de 1000×1000 a 50×50. Se seguía la expresión $\overline{E} = \left(\frac{1}{n}\sum_i^n E_i^p\right)^{1/p}$. También aplica a todos los calculados analíticamente.

Estos son los mapas de ganancia de energía de cada ventana:

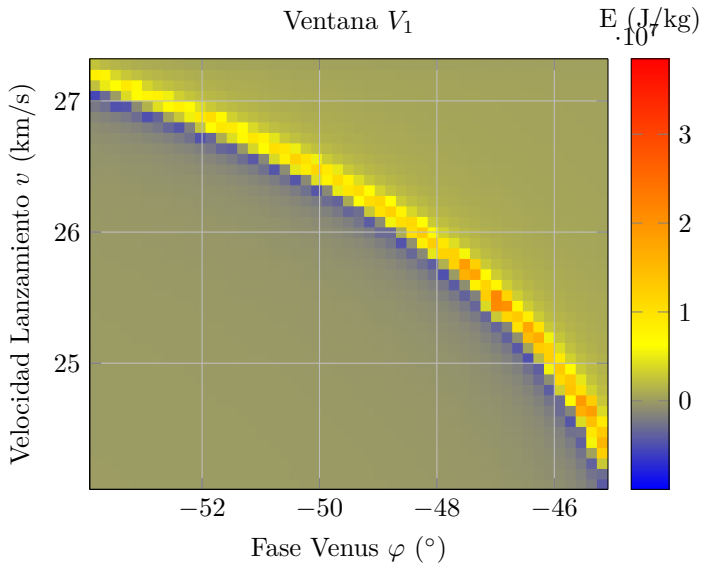


Figura 2: Mapa de energía de la ventana V_1.

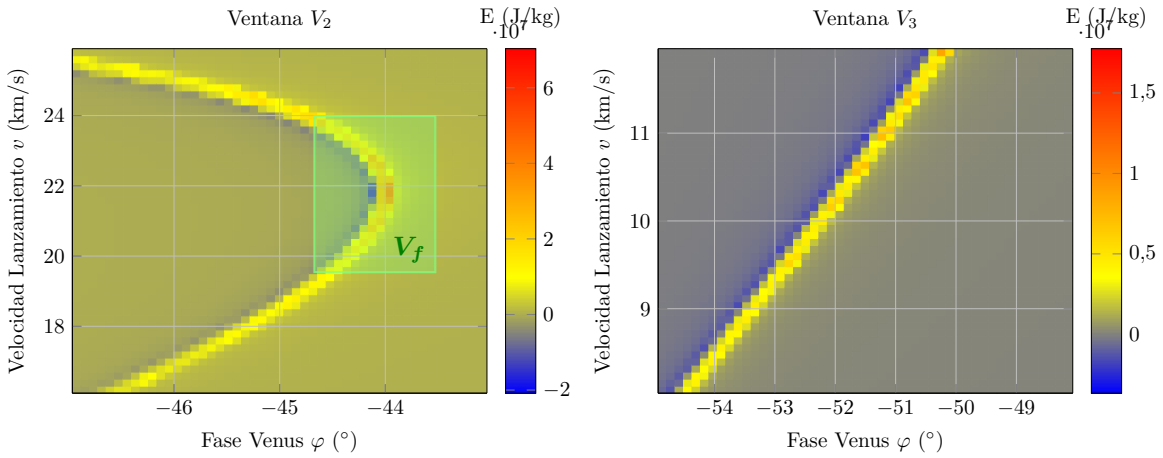


Figura 3: Mapa de energía de la ventanas V_2 y V_3.

Dado que en la ventana V_2 se llega a un máximo mayor que en las otras dos, el rango de valores a estudiar mediante la simulación numérica queda en la ventana V_f, con intervalos $v \in [19{,}5, 24]$ y $\varphi \in [-44{,}7, -43{,}5]$.

Con esos intervalos obtenemos por el método numérico:

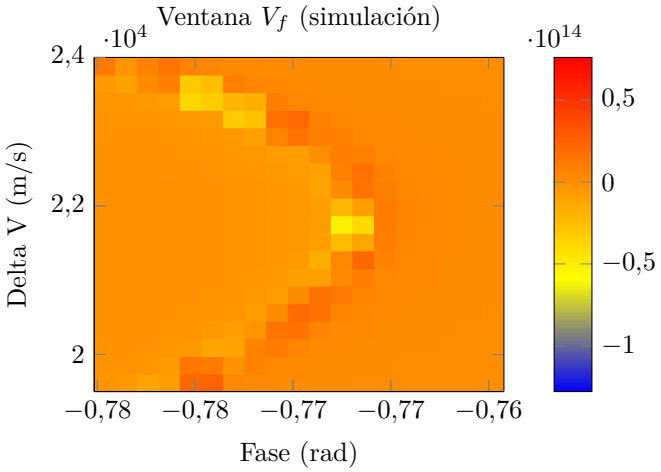


Figura 4: Mapa de energía mediante simulación.

Referencias

García, A. F. (2016). El problema de Kepler. http://www.sc.ehu.es/sbweb/fisica3/celeste/kepler/kepler.html

Subías, J. (2025). Apuntes de Mecánica Teórica y Analítica.

Weber, B. (2025, noviembre). Notes for Orbital Mechanics - Sphere of Influence. https://orbital-mechanics.space/interplanetary-maneuvers/sphere-of-influence.html